Assignment 3: Image Generation

Fuyuan Sun | Applied Generative AI

Implementation and training

The required PyTorch GAN is implemented in helper_lib. The generator maps 100-dimensional standard-normal noise through a fully connected layer to 128 x 7 x 7, then uses 4 x 4 transposed convolutions (stride 2, padding 1) with 128-to-64 and 64-to-1 channels. The first upsampling layer has BatchNorm and ReLU; the output uses Tanh.

The discriminator applies 4 x 4 convolutions (stride 2, padding 1) with 1-to-64 and 64-to-128 channels, LeakyReLU(0.2) after each convolution, BatchNorm after the second, and a flattened linear output followed by Sigmoid.

Training completed 5 full epochs on all 60,000 MNIST training images, normalized to [-1,1]. Batch size: 128; seed: 42; separate Adam optimizers: learning rate 0.0002, betas (0.5,0.999); device: CPU. Binary cross-entropy trains D on real images and detached fake images; the non-saturating G objective targets real labels. D parameters are frozen during the G update, with BatchNorm remaining in training mode. No test-set tuning was used.

Epoch	D loss	G loss	Training images
1	0.1876	2.2274	60000
2	0.1777	2.4440	60000
3	0.1780	2.4889	60000
4	0.1911	2.4352	60000
5	0.2003	2.4221	60000

Deployment and verification

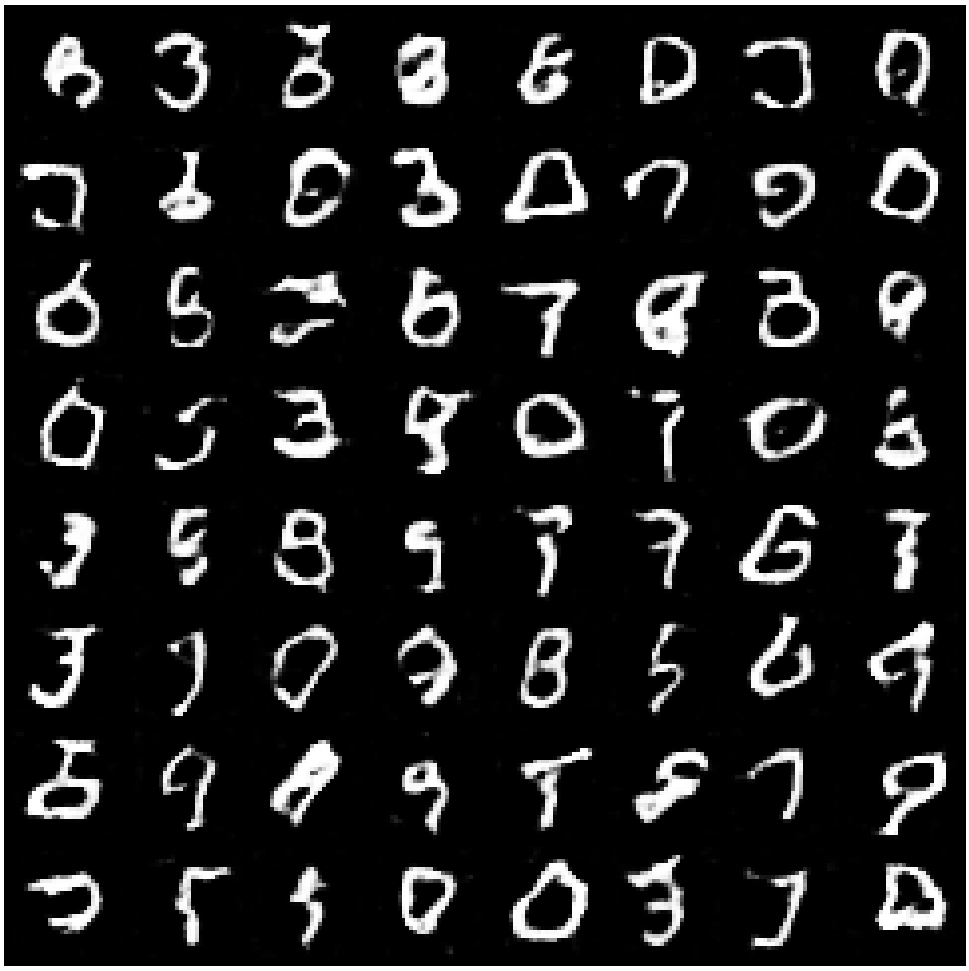
The existing FastAPI application now exposes GET /gan/info and POST /gan/generate. The POST body accepts num_samples (1-64) and seed; the response is an image/png grid. Existing text-generation, embedding and CNN-classification routes are preserved. Trained inference weights are included in the Docker image; the server does not download MNIST or train during startup.

Local validation: 26 automated tests passed, including prior assignments and new GAN tests. Real HTTP checks verified model readiness, a valid PNG response, and identical output for repeated requests with the same seed. Docker verification status and evidence are documented in README and the GitHub Actions run.

GitHub: assignment1-fastapi / assignment3-gan

Generated examples and interpretation

Fixed seed 42, 64 independently sampled noise vectors; checkpoint after epoch 5. Each source image is 28 x 28 pixels. The grid is enlarged here for readability. These are generated images, not training images.



Quality is assessed visually: recognizable digit-like shapes coexist with distorted or ambiguous samples. This unconditional model cannot choose a particular digit, and no quantitative quality score or classification accuracy is claimed. GAN losses are optimization diagnostics, not image-quality metrics. The final completed checkpoint is used without selecting on test data.

Reproduce generation

docker build -t assignment3-fastapi .
docker run --rm -p 8000:80 assignment3-fastapi
Open http://127.0.0.1:8000/docs and POST /gan/generate:
{"num_samples":16,"seed":42}

Prepared with OpenAI Codex assistance. The following pages contain all seven conceptual answers and calculations.

Assignment 3: Building Blocks of GAN

Question 1

For a transposed convolution with dilation 1, each spatial output dimension is:

$O = (I - 1)S - 2P + (K - 1) + OP + 1$
$= (I - 1)S - 2P + K + OP.$

Substitute $I = 8$, $K = 4$, $S = 2$, $P = 1$, $OP = 1$:

$O = (8 - 1) \times 2 - 2 \times 1 + 4 + 1$
$= 14 - 2 + 4 + 1 = 17.$

The spatial output is **17 x 17**.

Question 2

Keeping the values in Question 1 fixed except for $S = 3$:

$O = (8 - 1) \times 3 - 2 \times 1 + 4 + 1$
$= 21 - 2 + 4 + 1 = 24.$

The output changes from **17 x 17 to 24 x 24**. In general, increasing stride from 2 to 3 increases each output dimension by $I - 1$ (7 in this example).

Question 3

With dilation 1:

$O = (I - 1)S - 2P + K + OP.$

Apply this separately to height and width. More generally, with dilation D:

$O = (I - 1)S - 2P + D(K - 1) + OP + 1.$

Here I is input size, K kernel size, S stride, P padding and OP output padding.

Question 4

Choose **kernel size 2 x 2, stride 2, padding 0, output padding 0**, dilation 1:

$O = (16 - 1) \times 2 - 0 + 2 + 0 = 30 + 2 = 32.$

Thus a 16 x 16 map becomes 32 x 32. A 4 x 4 kernel with stride 2 and no padding would give 34 x 34, so it does not meet this question's condition.

Question 5

The mini-batch is [6, 8, 10, 6], with N = 4.

Mean: mu = (6 + 8 + 10 + 6) / 4 = 30 / 4 = 7.5.
Deviations: [-1.5, 0.5, 2.5, -1.5].
Squared deviations: [2.25, 0.25, 6.25, 2.25].
Batch variance: var = (2.25 + 0.25 + 6.25 + 2.25) / 4 = 11 / 4 = 2.75.
Standard deviation: sqrt(2.75) = sqrt(11) / 2, approximately 1.658312.

Without scale gamma or shift beta, normalized values are (x - mu) / sqrt(var):

[-3/sqrt(11), 1/sqrt(11), 5/sqrt(11), -3/sqrt(11)]

Approximately **[-0.904534, 0.301511, 1.507557, -0.904534]**.

Their mean is 0 and their variance is 1. BatchNorm uses the population-style variance (divide by N) for training-time normalization, not the sample variance (divide by N - 1). Real implementations add epsilon inside the square root: (x - mu)/sqrt(var + epsilon), so the resulting variance is slightly below 1. The values above use epsilon = 0 as the question requests unit variance.

Question 6

ReLU(x) = max(0, x): x for x >= 0, and 0 for x < 0.

LeakyReLU(x) = x for x >= 0, and alpha*x for x < 0, where 0 < alpha < 1.
For this assignment, alpha = 0.2.

The key difference is the negative-input slope: ReLU has derivative 0 there, whereas LeakyReLU has derivative alpha. Both have positive-input slope 1. At x = 0 they have a kink; implementations choose a derivative convention.

Question 7

A ReLU neuron whose pre-activation stays negative produces zero output and zero local gradient. Its incoming parameters may therefore stop receiving a useful learning signal (the dying-ReLU problem).

LeakyReLU keeps a small nonzero slope on the negative side, allowing gradients to propagate and making such units less likely to become permanently inactive. This can help optimization in deep networks and GAN discriminators. It does not guarantee better results for every network or dataset.

Prepared with OpenAI Codex assistance. Review and understand the derivations and the code before submission.